

Speedhawk Architecture Overview

Speedhawk is structured as three main applications that communicate through two shared services. The three applications are the frontend, the backend API, and the worker service. The shared services are PostgreSQL and Redis. This separation keeps the system scalable, easier to maintain, and closer to real production architecture.

1. Frontend (Client)

The frontend is the user-facing React application. It displays the landing page, authentication screens, URL submission form, dashboards, and performance reports. The frontend only communicates with the backend API using HTTP requests.

2. Backend API (Server)

The backend is a Node.js + Express API that acts as the main controller of the system. It handles authentication, validates submitted URLs, stores audit records in PostgreSQL, and sends audit jobs into a Redis queue. It also returns audit results to the frontend.

3. Worker Service

The worker is responsible for heavy background processing. It listens for jobs in Redis, runs Lighthouse and Puppeteer audits, parses the performance metrics, and saves the results back into PostgreSQL.

4. PostgreSQL Database

PostgreSQL stores all persistent data including users, audits, metrics, and optimization suggestions. Both the backend API and the worker service connect to PostgreSQL.

5. Redis Queue

Redis is used as a job queue system using BullMQ. The backend API pushes audit jobs into the queue and the worker consumes those jobs to perform the actual performance analysis.

End-to-End Flow

1. User submits a URL from the frontend.
2. Frontend sends request to backend API.
3. Backend stores a pending audit in PostgreSQL.
4. Backend pushes a job into the Redis queue.
5. Worker consumes the job and runs Lighthouse audit.
6. Worker saves metrics and suggestions into PostgreSQL.
7. Frontend requests results from the backend and displays them on the dashboard.